

- *Base de Données NoSQL : MongoDB* -

TP 2 : Map/Reduce

Partie I : Introduction & Exemples d'utilisation

Map-Reduce est un paradigme de traitement parallèle de données permettant de condenser de gros volumes de données en résultats agrégés utiles. Pour effectuer des opérations de Map-Reduce, MongoDB fournit la commande **mapReduce** :

```
db.<collection_name>.mapreduce(<map_fct>, <reduce_fct>, <output>)
```

Les deux fonctions *Map()* et *Reduce()* doivent respecter les contraintes suivantes :

➤ Paramètre 1 : La fonction Map()

- Appliquer sur chaque document de la collection.
- Ne prend aucun paramètre, on accède à l'objet analysé via l'opérateur **this**.
- Peut émettre des couples via la fonction **emit(key, value)** autant de fois que nécessaire dans la fonction.

➤ Paramètre 2 : La fonction Reduce()

- Prend deux paramètres key et values (tableau des valeurs de la clé).

➤ Paramètre 3 : Format de sortie

On doit passer un troisième paramètre via l'option **out** qui spécifie l'emplacement du résultat :

- **{out:{ inline : 1}}** : Si on veut juste voir le résultat, sans le stocker.
- **{out : nom_collection}** : définit la collection dans laquelle le résultat sera stocké.

Pour plus d'informations, consulter le manuel de MongoDB via le lien suivant:

<https://docs.mongodb.com/manual/core/map-reduce/>

⇒ **Données de test :**

Récupérer le fichier « *sportifs.bson* » et importer le avec la commande **mongoimport**.

⇒ **Exemple 1**

Dans l'exemple qui suit, nous allons chercher à calculer le nombre d'hommes et de femmes. Tout d'abord, on définit la fonction *map()*, qui émettra un couple (sexe, 1), pour chaque sportif.

```
var map1 = function() {  
    emit(this.Sexe, 1)  
};
```

Ensuite, nous définissons la fonction *reduce()*, qui fera la somme des valeurs (qui seront tous des 1).

```
var red1 = function(key, values) {  
    return Array.sum(values);  
};
```

Ensuite, nous faisons appel à la fonction **mapReduce()** sur notre collection.

```
db.Sportifs.mapReduce(  
    map1,  
    red1,  
    { out: { inline: 1 } }  
)
```

On remarque qu'on a **m** et **M**. On peut résoudre ce problème en faisant une transformation (en majuscule ou en minuscule) dans la fonction *map()*.

```
var map1 = function() {  
    emit(this.Sexe.toUpperCase(), 1)  
}
```

Au final, on peut aussi tout déclarer dans la fonction *mapReduce()*, en utilisant le principe des fonctions anonymes.

```
db.Sportifs.mapReduce(  
    function() {  
        emit(this.Sexe.toUpperCase(), 1)  
    },  
    function(key, values) {  
        return Array.sum(values);  
    },  
    { out: { inline: 1 } }
```

```
{ out: { inline: 1 } }  
)
```

⇒ Exemple 2

Si on cherche à calculer le nombre de sportifs jouant pour chaque sport, il faut émettre des couples (sport, 1) pour chaque sport joué, pour chaque sportif. Mais il faut d'une part prendre en compte qu'il existe des sportifs dans la base, qui ne jouent aucun sport. Et d'autre part que pour certains sportifs, il n'y a qu'un sport, et que dans ce cas, nous avons juste une chaîne et non un tableau.

```
db.Sportifs.mapReduce(  
  function() {  
    if (this.Sports) { // le sportif joue  
      if (typeof this.Sports.Jouer != "string") {  
        for (sp of this.Sports.Jouer) {  
          emit(sp, 1)  
        }  
      } else {  
        emit(this.Sports.Jouer, 1)  
      }  
    }  
  },  
  function(key, values) {  
    return Array.sum(values);  
  },  
  { out: { inline: 1 } }  
)
```

Partie II : Application

I. Importer le fichier « *salles_sport.bson* ».

II. En utilisant le paradigme **Map-Reduce** :

1. Calculer le nombre de salles de sports pour chaque ville.
2. Calculer le nombre de séances pour chaque jour de la semaine.
3. De même pour chaque sport.
4. Calculer la superficie moyenne des salles de sports, pour chaque ville
5. Calculer pour chaque sport, le nombre de séance pour chaque jour de la semaine